



SCHOOL OF COMPUTING

Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**Library Management System**” is submitted by

G.Rambabu	(25B21AI347)
B.Taraka Ram	(25B21AI092)
G.Gayathri	(25B21AI348)
Ch.Shiva Ram Teja	(25B21AI220)

In partial fulfilment of the requirements for the award of the B.Tech. degree in Artificial Intelligence & Machine Learning for the academic year 2026-2027.

Project Guide

Mrs. G.L. Narasamba Vanguri, M. Tech, (Ph.D.)
Assistant Professor,
Department of AIML

Head of the Department

Dr K. Naga Bhargavi, M. Tech, Ph.D.,
Associate Professor,
Department of AIML

LIBRARY MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award
of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

G.Rambabu (25B21AI347)

B.Taraka Ram (25B21AI092)

G.Gayathri (25B21AI348)

Ch.Shiva Ram Teja (25B21AI220)

Under the esteemed guidance of

G L NARASAMBA VANGURI

M.Tech(Ph.D)

Assistant Professor

Department of AI&ML

ADITYA UNIVERSIT



A D I T Y A
U N I V E R S I T Y

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



ADITYA UNIVERSITY

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027



ADITYA UNIVERSITY

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



A D I T Y A
UNIVERSITY

CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “library Management System” is being submitted by gG.Gayathri(Roll no.25B11AI348),G.Rambabu(Roll no.25B11AI347), B.Taraka Ram (Roll No. 25B11AI092) and Ch.Siva Rama Teja(Roll No. 25B11AI220) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

G.L.Narasamba.v, M. Tech, (Ph.D.)
Assistant professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Assistant Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “ LIBRARY MANAGEMENT SYSTEM” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mr. T. Srinivasulu, M. Tech, (Ph.D.), Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

G.Rambabu (25B21AI347)

B.Taraka Ram (25B21AI092)

G.Gayathri (25B21AI348)

Ch.Shiva Ram (25B21AI220)

Teja

)

Date: 14 September 2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide G.L.Narasamba.v, M. Tech, (Ph.D.), Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Head of the Department of the Artificial Intelligence and Machine Learning for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

G.Rambabu	(25B11AI347)
B.Taraka Ram	(25B11AI092)
G.Gayathri	(25B21AI348)
Ch.Shiva Ram Teja	(25B11AI220)

ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

DBMS CAPSTONE PROJECT

Project Documentation Report

Project Title: library Management System

Submitted by:

S.No	Student Name	Roll Number
1	G.Rambabu	25B11AI347
2	G,Gayathri	25B11AI348
3	B.Taraka Ram	25B11AI092
4	Ch. Shiva Ram Teja	25B11AI220

Course: Database Management System (DBMS) Course Code: DBMS

Faculty Coordinator:

Academic Year: 2026–27

Date of Submission: 14 September 2026

GitHub: <https://github.com/gollapalligayathri07-gif/LibraryManagement-System.git>

1. ABSTRACT

The Library Management System is a database project designed to manage and organize library information in a structured and efficient manner. The system focuses on maintaining details of books, authors, publishers, members, loans, and fines. The database addresses common problems such as data redundancy, difficulty in retrieving information, inconsistent records, and improper management of borrowing and fine details.

The main purpose of the project is to provide a normalized relational database that stores library information accurately and maintains relationships between different records. The system consists of six main entities: Book, Author, Publisher, Member, Loan, and Fine, along with the Book_Author junction table for managing the relationship between books and authors. SQL is used for database creation, data insertion, retrieval, updating, and deletion, along with primary keys, foreign keys, unique constraints, and other integrity rules.

SQL operations such as CREATE, INSERT, UPDATE, DELETE, SELECT, joins, aggregate functions, GROUP BY, HAVING, subqueries, views, and set operations are implemented. The database design is normalized up to Third Normal Form (3NF) to reduce redundancy and improve data consistency. The project provides a structured foundation for managing library operations and retrieving required information efficiently.

TABLE OF CONTENTS

Chapter	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Project Documentation Report	iv
1	ABSTRACT	v
	List of Figures	vii
	List of Tables	vii
2	INTRODUCTION	1
	2.1 Background	1
	2.2 Purpose of the Database	1
3	PROBLEM STATEMENT	1
4	OBJECTIVES	1
5	TOOLS & TECHNOLOGIES USED	2
6	DATABASE DESIGN	2
	6.1 Identified Entities	2
	6.2 Entity Attributes	2
	6.3 ER Diagram	4
	6.4 Relationships & Cardinality	4
	6.5 Relational Database Schema	4
	6.6 Normalization	6
7	DATABASE IMPLEMENTATION	8
	7.1 DDL — Database and Table Creation	8
	7.2 Sample Data — DML.	10
	7.3 Constraints & Data Integrity	10
	7.4 SQL Operations & Queries	10
	7.5 Sample Outputs / Screenshots	12
8	MODULES & FUNCTIONALITIES	14
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	14
10	ENHANCEMENTS & FUTURE IMPROVEMENTS	15
11	CONCLUSION	15
12	REFERENCES	15
	APPENDIX	16
	A. Complete SQL Script	16
	B. Additional Screenshots / Outputs	16

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
6.3	ER Diagram	4
Figure 1	Customer records / basic SELECT output	12
Figure 2	Active savings accounts / filtering output	12
Figure 3	Aggregate or GROUP BY output	13
Figure 4	Join or subquery output	13

LIST OF TABLES

Table No.	Title of the Table	Page No.
5	TOOLS & TECHNOLOGIES USED	2
6.1	Identified Entities	2
6.2	Entity Attributes	2
6.4	Relationships & Cardinality	4
6.5	Relational Database Schema	4
7.2	Sample Data — DML.	10
7.3	Constraints & Data Integrity	10
7.4	SQL Operations & Queries	10
8	MODULES & FUNCTIONALITIES	14
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	14

2. INTRODUCTION

2.1 Background

The **Library Management System** is a database project designed to manage the day-to-day operations of a library in an organized and efficient manner. A library handles different types of information, including publishers, authors, books, members, book borrowing records, and fines. Managing this information manually can make it difficult to maintain accurate records and retrieve information quickly.

The proposed database uses **PostgreSQL** to store and manage the library's data. The database is organized into related tables such as **Publisher, Author, Book, Book_Author, Member, Loan, and Fine**. Primary keys and foreign keys are used to establish relationships between the tables and maintain data consistency.

The project also demonstrates important **DBMS concepts** such as DDL, DML, DQL, CRUD operations, primary keys, foreign keys, constraints, joins, aggregate functions, and views

2.2 Purpose of the Database

The main purpose of the Library Management System database is to provide a structured system for storing, managing, and retrieving library information.

The database is designed to:

- Store information about **books, authors, and publishers**.
- Maintain **member details**.
- Record **book loans and returns**.
- Manage **fine information** related to loans.
- Establish relationships between books and authors using the **Book_Author** table.
- Reduce data duplication through proper database organization.
- Maintain data consistency using **primary keys, foreign keys, and constraints**.
- Retrieve useful information using **SQL queries, joins, and aggregate functions**.
- Demonstrate practical implementation of **DBMS concepts using PostgreSQL**.

3. PROBLEM STATEMENT

In a library, information about books, authors, publishers, members, loans, and fines may become difficult to manage when records are maintained manually or in separate files. Such an approach can lead to **data redundancy, inconsistent information, difficulty in retrieving records, and lack of proper integrity**

checks. Managing book availability, member borrowing details, loan dates, return dates, and fine information manually can also make it difficult to maintain accurate and updated records.

The proposed **Library Management System** addresses these limitations by using a structured relational database. The database stores related information in separate tables and connects them using **primary keys and foreign keys**. Constraints such as **UNIQUE, NOT NULL, and referential integrity** help maintain accurate and consistent records. SQL operations can be used to efficiently insert, update, delete, and retrieve information. This provides a systematic way to manage library records and reduces redundancy and data inconsistencies.

4. OBJECTIVES

1. To design a normalized relational database for the selected domain with at least six entities.
2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.
3. To efficiently store, retrieve, update, and manage library information using SQL operations.
4. To establish proper relationships between books, authors, publishers, members, loans, and fines.
5. To reduce data redundancy and maintain data consistency through normalization up to Third Normal Form (3NF).

5. TOOLS & TECHNOLOGIES USED

Category	Tool / Technology Used	Version / Remarks
Database Platform	dbfiddle.dev	Used to create and execute SQL queries
Query / Admin Tool	SQL	Used for DDL, DML and query operations
ER Diagram Tool	Diagrams.net (draw.io)	Used to create the ER diagram
Documentation	Microsoft Word	Used to prepare the project report
Version Control	GitHub	Used to store and manage project files
Database Design	Relational Database Model	Used for designing tables and relationships

6. DATABASE DESIGN

6.1 Identified Entities

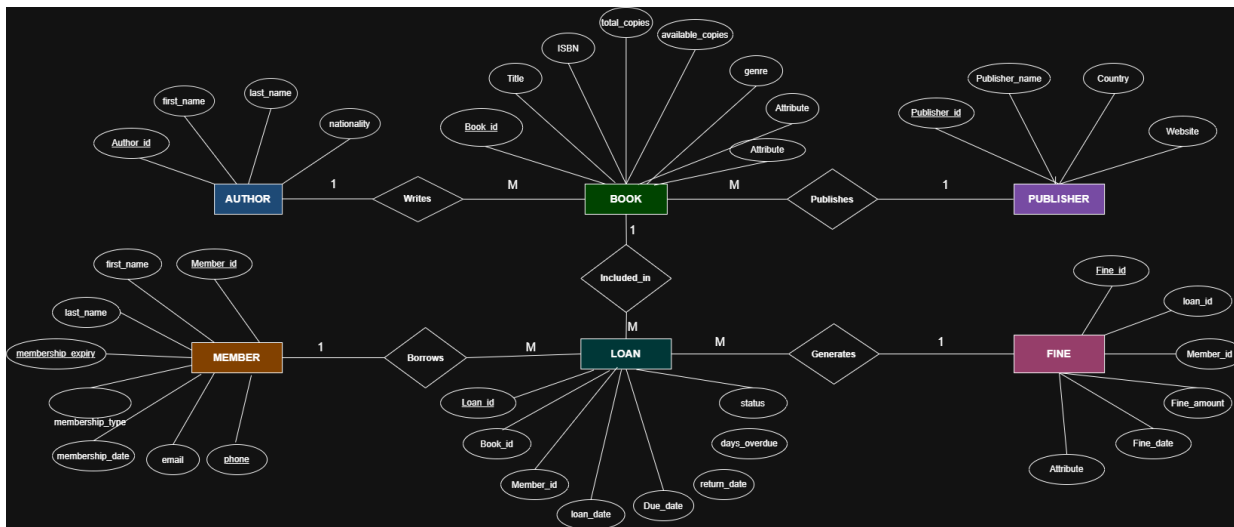
S.No	Entity Name	Description
1	Book	Stores information about books available in the library, such as title, ISBN, genre, and number of copies.
2	Author	Stores details of authors who have written the books, including their name and nationality.
3	Publisher	Stores information about publishers, including publisher name, country, and website.
4	Member	Stores details of registered library members, including their name, email, membership type, and dates.
5	Loan	Stores information about books borrowed by members, including loan date, due date, and return date.
6	Fine	Stores fine details related to loans, including fine amount, fine date, and payment status.

6.2 Entity Attributes

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
BOOK	Book_id	INT	PK	simple
Book	title	VARCHAR(200)	—	simple
Book	isbn	VARCHAR(20)	—	simple
Book	Publication_year	YEAR	—	simple
Book	Authors	TEXT	—	Multi-valued
Book	Total_copies	INT	—	Simple
Book	Available_copies	INT	—	Derived
Book	genre	VARCHAR(20)	—	simple
AUTHOR	Author_id	INT	pk	simple
Author	First_name	VARCHAR(20)	—	Composite part
Author	Last_name	VARCHAR(20)	—	Composite part
Author	nationality	VARCHAR(20)	—	simple
PUBLISHER	Publisher_id	INT	PK	Simple
Publisher	Publisher_name	VARCHAR(20)	—	simple
Publisher	Country	VARCHAR(20)	—	Simple
Publisher	website	VARCHAR(20)	—	Simple
MEMBER	Member_id	INT	PK	Simple

Member	First_name	VARCHAR(20)	—	Composite part
Member	Last_name	VARCHAR(20)	—	Composite part
Member	Email	VARCHAR(20)	—	simple
Member	Membership_type	VARCHAR(20)	—	Simple
Member	Membership_date	DATE	—	Simple
Member	Membership_expiry	DATE	—	Simple
Member	phone	INT	—	simple
LOAN	Loan_id	INT	PK	Simple
Loan	Book_id	INT	FK	Simple
Loan	Member_id	INT	FK	Simple
Loan	Loan_date	DATE	—	Simple
Loan	Due_date	DATE	—	Simple
Loan	Return_date	DATE	—	Simple
Loan	Days_overdue	INT	—	simple
Loan	status	VARCHAR(20)	—	Simple
FINE	Fine_id	INT	PK	Simple
Fine	Loan_id	INT	FK	Simple
Fine	Member_id	INT	FK	Simple
Fine	Fine_amount	DECIMAL(8,2)	—	Simple
Fine	Fine_date	DATE	—	Simple

6.2ER Diagram



6.3 Relationships & Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)	Participation
Book	Written by	Author	M:N	partial
Book	Published by	Publisher	N:1	Partial
Member	Borrows	Book	M:N	Partial
Member	Pays	Fine	1:1	Partial
Loan	has	fine	1:N	partial

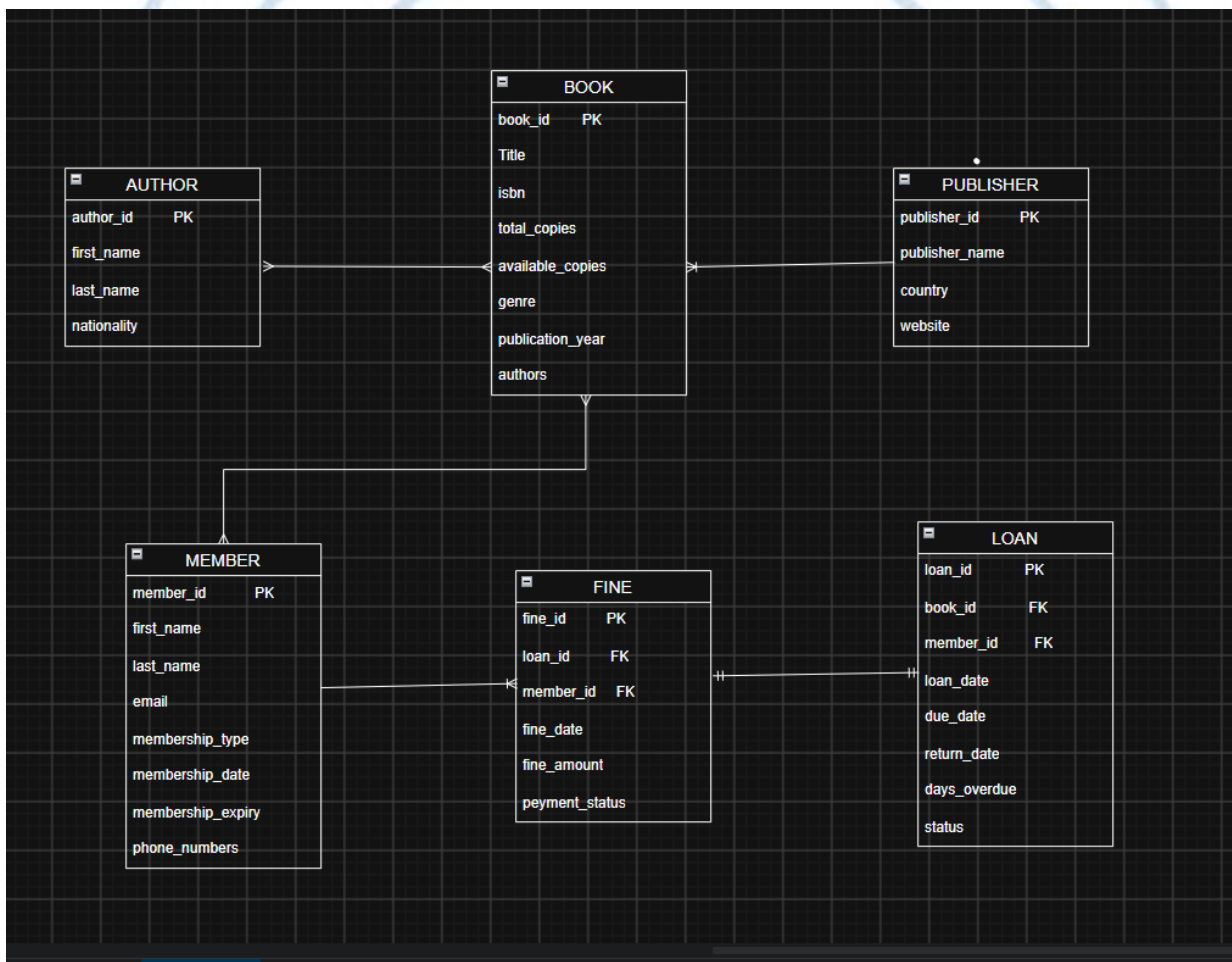
6.4 Relational Database Schema

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Book	book_id	INT	PK	PRIMARY KEY
Book	title	VARCHAR(200)	—	NOT NULL
Book	isbn	VARCHAR(20)	—	UNIQUE, NOT NULL
Book	total_copies	INT	—	—
Book	available_copies	INT	—	Derived
Book	genre	VARCHAR(50)	—	—
Book	publication_year	YEAR	—	—

Book	authors	TEXT	—	Multi-valued
Author	author_id	INT	PK	PRIMARY KEY
Author	first_name	VARCHAR(50)	—	—
Author	last_name	VARCHAR(50)	—	—
Author	nationality	VARCHAR(50)	—	—
Publisher	publisher_id	INT	PK	PRIMARY KEY
Publisher	publisher_name	VARCHAR(100)	—	NOT NULL
Publisher	country	VARCHAR(50)	—	—
Publisher	website	VARCHAR(200)	—	—
Member	member_id	INT	PK	PRIMARY KEY
Member	first_name	VARCHAR(50)	—	—
Member	last_name	VARCHAR(50)	—	—
Member	email	VARCHAR(100)	—	UNIQUE
Member	membership_type	VARCHAR(30)	—	—
Member	membership_date	DATE	—	—
Member	membership_expiry	DATE	—	—
Member	phone_numbers	TEXT	—	Multi-valued
Loan	loan_id	INT	PK	PRIMARY KEY
Loan	book_id	INT	FK	FK → Book
Loan	member_id	INT	FK	FK → Member
Loan	loan_date	DATE	—	NOT NULL
Loan	due_date	DATE	—	—
Loan	return_date	DATE	—	—
Loan	days_overdue	INT	—	Derived
Loan	status	VARCHAR(20)	—	—

Fine	fine_id	INT	PK	PRIMARY KEY
Fine	loan_id	INT	FK	FK → Loan
Fine	member_id	INT	FK	FK → Member
Fine	fine_amount	DECIMAL(8,2)	—	—
Fine	fine_date	DATE	—	—
Fine	payment_status	VARCHAR(20)	—	—

Schema Diagram



6.5 Normalization

UNF → 1NF → 2NF → 3NF

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Tables
UNF	Data may contain repeating groups and multiple values in a single record.	Repeating-group and insertion problems identified.	Library_Records
1NF	All attributes contain atomic values and repeating groups are removed.	Removes repeating groups and non-atomic values.	Book, Author, Publisher, Member, Loan, Fine, Book_Author
2NF	Relation is in 1NF and all non-key attributes are fully dependent on the complete primary key.	Removes partial dependency and associated update anomalies.	Book, Author, Publisher, Member, Loan, Fine, Book_Author
3NF	Relation is in 2NF and non-key attributes depend only on the primary key, with no transitive dependency.	Removes transitive dependency and reduces redundancy.	Book, Author, Publisher, Member, Loan, Fine, Book_Author

7. DATABASE IMPLEMENTATION

7.1 DDL — Database and Table Creation

```
CREATE TABLE Publisher (  
    publisher_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    publisher_name VARCHAR(100) NOT NULL,  
    country VARCHAR(50),  
    website VARCHAR(200)  
);
```

```
CREATE TABLE Author (  
    author_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50),  
    nationality VARCHAR(50)  
);
```

```
CREATE TABLE Book (  
    book_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    isbn VARCHAR(20) UNIQUE NOT NULL,  
    publication_year INT,  
    total_copies INT NOT NULL CHECK (total_copies >= 0),  
    available_copies INT NOT NULL CHECK (available_copies >= 0),  
    genre VARCHAR(50),  
    publisher_id INT,  
  
    CONSTRAINT fk_book_publisher  
        FOREIGN KEY (publisher_id)  
        REFERENCES Publisher(publisher_id),  
  
    CONSTRAINT chk_available_copies  
        CHECK (available_copies <= total_copies)  
);
```

```
CREATE TABLE Member (  
    member_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50),  
    email VARCHAR(100) UNIQUE NOT NULL,  
    membership_type VARCHAR(30),  
    membership_date DATE,  
    membership_expiry DATE,  
    phone_numbers VARCHAR(20)  
);
```

```

CREATE TABLE Book_Author (
    book_id INT,
    author_id INT,

    PRIMARY KEY (book_id, author_id),

    CONSTRAINT fk_book_author_book
        FOREIGN KEY (book_id)
        REFERENCES Book(book_id),

    CONSTRAINT fk_book_author_author
        FOREIGN KEY (author_id)
        REFERENCES Author(author_id)
);

CREATE TABLE Loan (
    loan_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id INT NOT NULL,
    member_id INT NOT NULL,
    loan_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE,
    days_overdue INT DEFAULT 0,
    status VARCHAR(20) DEFAULT 'Borrowed',

    CONSTRAINT fk_loan_book
        FOREIGN KEY (book_id)
        REFERENCES Book(book_id),

    CONSTRAINT fk_loan_member
        FOREIGN KEY (member_id)
        REFERENCES Member(member_id),

    CONSTRAINT chk_loan_dates
        CHECK (due_date >= loan_date)
);

CREATE TABLE Fine (
    fine_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    loan_id INT NOT NULL,
    member_id INT NOT NULL,
    fine_amount DECIMAL(8,2) CHECK (fine_amount >= 0),
    fine_date DATE,
    payment_status VARCHAR(20) DEFAULT 'Pending',

    CONSTRAINT fk_fine_loan
        FOREIGN KEY (loan_id)

```

```
REFERENCES Loan(loan_id),  
  
CONSTRAINT fk_fine_member  
FOREIGN KEY (member_id)  
REFERENCES Member(member_id)  
);
```



Postgres

```
- CREATE TABLE Publisher (  
  publisher_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  publisher_name VARCHAR(100) NOT NULL,  
  country VARCHAR(50),  
  website VARCHAR(200)  
);  
  
- CREATE TABLE Author (  
  author_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  nationality VARCHAR(50)  
);  
  
- CREATE TABLE Book (  
  book_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  isbn VARCHAR(20) UNIQUE NOT NULL,  
  publication_year INT,  
  total_copies INT NOT NULL CHECK (total_copies >= 0),  
  available_copies INT NOT NULL CHECK (available_copies >= 0),  
  genre VARCHAR(50),  
  publisher_id INT,  
  
  CONSTRAINT fk_book_publisher  
    FOREIGN KEY (publisher_id)  
    REFERENCES Publisher(publisher_id),  
  
  CONSTRAINT chk_available_copies  
    CHECK (available_copies <= total_copies)  
);  
  
- CREATE TABLE Member (  
  member_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50),  
  email VARCHAR(100) UNIQUE NOT NULL,  
  membership_type VARCHAR(30),  
  membership_date DATE,  
  membership_expiry DATE,  
  phone_numbers VARCHAR(20)  
);  
  
- CREATE TABLE Book_Author (  
  book_id INT,  
  author_id INT,  
  
  PRIMARY KEY (book_id, author_id),  
  
  CONSTRAINT fk_book_author_book  
    FOREIGN KEY (book_id)  
    REFERENCES Book(book_id),  
  
  CONSTRAINT fk_book_author_author  
    FOREIGN KEY (author_id)  
    REFERENCES Author(author_id)  
);  
  
- CREATE TABLE Loan (  
  loan_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  book_id INT NOT NULL,  
  member_id INT NOT NULL,  
  loan_date DATE NOT NULL,  
  due_date DATE NOT NULL,  
  return_date DATE,  
  days_overdue INT DEFAULT 0,  
  status VARCHAR(20) DEFAULT 'Borrowed',  
  
  CONSTRAINT fk_loan_book  
    FOREIGN KEY (book_id)  
    REFERENCES Book(book_id),  
  
  CONSTRAINT fk_loan_member  
    FOREIGN KEY (member_id)  
    REFERENCES Member(member_id),  
  
  CONSTRAINT chk_loan_dates  
    CHECK (due_date >= loan_date)  
);  
  
- CREATE TABLE Fine (  
  fine_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  loan_id INT NOT NULL,  
  member_id INT NOT NULL,  
  fine_amount DECIMAL(8,2) CHECK (fine_amount >= 0),  
  fine_date DATE,  
  payment_status VARCHAR(20) DEFAULT 'Pending',  
  
  CONSTRAINT fk_fine_loan  
    FOREIGN KEY (loan_id)  
    REFERENCES Loan(loan_id),  
  
  CONSTRAINT fk_fine_member  
    FOREIGN KEY (member_id)  
    REFERENCES Member(member_id)  
);
```

Click Run or press: **shift + ENTER** to run code[+ Code](#) [+ Markdown](#)

7.2 Sample Data — DML.

S.No	Table Name	No. of Records Inserted	Screenshot Reference
1	Publisher	4	Fig. 4.2
2	Author	5	Fig. 4.2
3	Book	5	Fig.4.2
4	Book_Author	5	Fig.4.2
5	Member	4	Fig.4.2
6	Loan	4	Fig.4.2
7	Fine	1	Fig.4.2

```
--publisher
INSERT INTO Publisher
(publisher_name, country, website)
VALUES
('person', 'USA', 'https://www.pearson.com'),
('McGraw Hill', 'USA', 'https://www.mheducation.com'),
('OReilly Media', 'USA', 'https://www.oreilly.com'),
('Wiley', 'USA', 'https://www.wiley.com');
```



```
--Author
INSERT INTO Author
(first_name, last_name, nationality)
VALUES
```

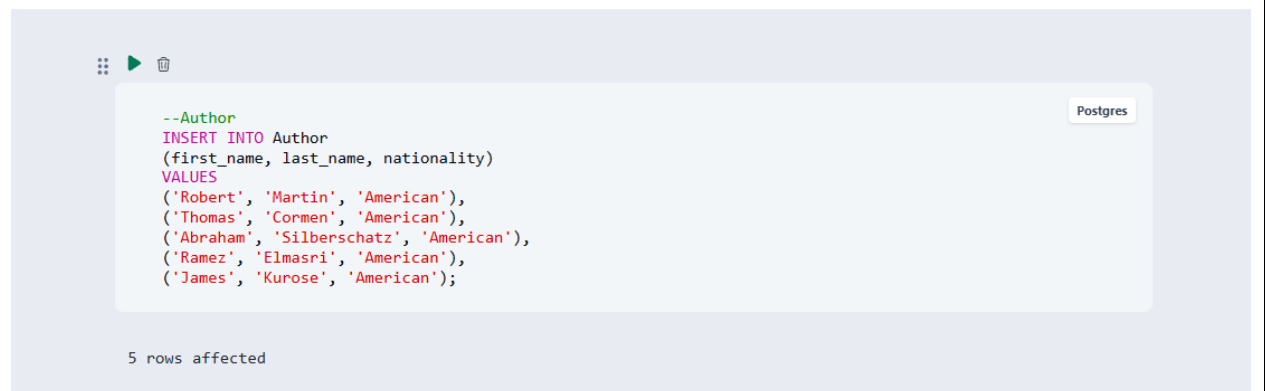
```
--publisher
INSERT INTO Publisher
(publisher_name, country, website)
VALUES
('Pearson', 'USA', 'https://www.pearson.com'),
('McGraw Hill', 'USA', 'https://www.mheducation.com'),
('OReilly Media', 'USA', 'https://www.oreilly.com'),
('Wiley', 'USA', 'https://www.wiley.com');
```



```
--publisher
INSERT INTO Publisher
(publisher_name, country, website)
VALUES
('Pearson', 'USA', 'https://www.pearson.com'),
('McGraw Hill', 'USA', 'https://www.mheducation.com'),
('OReilly Media', 'USA', 'https://www.oreilly.com'),
('Wiley', 'USA', 'https://www.wiley.com');
```

4 rows affected

```
--Author
INSERT INTO Author
(first_name, last_name, nationality)
VALUES
('Robert', 'Martin', 'American'),
('Thomas', 'Cormen', 'American'),
('Abraham', 'Silberschatz', 'American'),
('Ramez', 'Elmasri', 'American'),
('James', 'Kurose', 'American');
```



```
--Author
INSERT INTO Author
(first_name, last_name, nationality)
VALUES
('Robert', 'Martin', 'American'),
('Thomas', 'Cormen', 'American'),
('Abraham', 'Silberschatz', 'American'),
('Ramez', 'Elmasri', 'American'),
('James', 'Kurose', 'American');
```

5 rows affected

```
--Book
INSERT INTO Book
(title, isbn, publication_year, total_copies,
available_copies, genre, publisher_id)
VALUES
('Clean Code', '9780132350884', 2008, 5, 4, 'Programming', 1),
('Introduction to Algorithms', '9780262046305', 2022, 4, 3, 'Algorithms', 2),
('Database System Concepts', '9781260084504', 2019, 6, 5, 'Database', 2),
('Fundamentals of Database Systems', '9780137502523', 2020, 5, 5, 'Database', 1),
('Computer Networking', '9780136681557', 2021, 4, 4, 'Networking', 1);
```

```
--Book
INSERT INTO Book
(title, isbn, publication_year, total_copies,
available_copies, genre, publisher_id)
VALUES
('Clean Code', '9780132350884', 2008, 5, 4, 'Programming', 1),
('Introduction to Algorithms', '9780262046305', 2022, 4, 3, 'Algorithms', 2),
('Database System Concepts', '9781260084504', 2019, 6, 5, 'Database', 2),
('Fundamentals of Database Systems', '9780137502523', 2020, 5, 5, 'Database', 1),
('Computer Networking', '9780136681557', 2021, 4, 4, 'Networking', 1);
```

5 rows affected

+ Code + Markdown

```
INSERT INTO Book_Author
(book_id, author_id)
VALUES
(1, 1),
(2, 2),
(3, 3),
(4, 4),
(5, 5);
```

```
INSERT INTO Book_Author
(book_id, author_id)
VALUES
(1, 1),
(2, 2),
(3, 3),
(4, 4),
(5, 5);
```

5 rows affected

```
INSERT INTO Member
(first_name, last_name, email, membership_type,
membership_date, membership_expiry, phone_numbers)
VALUES
('Rahul', 'Kumar', 'rahul@gmail.com',
'Student', '2026-06-01', '2027-06-01', '9876543210'),

('Priya', 'Sharma', 'priya@gmail.com',
'Student', '2026-06-05', '2027-06-05', '9876543211'),

('Arjun', 'Reddy', 'arjun@gmail.com',
'Faculty', '2026-06-10', '2027-06-10', '9876543212'),

('Sneha', 'Patel', 'sneha@gmail.com',
'Student', '2026-06-15', '2027-06-15', '9876543213');
```

```
INSERT INTO Member
  (first_name, last_name, email, membership_type,
  membership_date, membership_expiry, phone_numbers)
VALUES
  ('Rahul', 'Kumar', 'rahul@gmail.com',
  'Student', '2026-06-01', '2027-06-01', '9876543210'),
  ('Priya', 'Sharma', 'priya@gmail.com',
  'Student', '2026-06-05', '2027-06-05', '9876543211'),
  ('Arjun', 'Reddy', 'arjun@gmail.com',
  'Faculty', '2026-06-10', '2027-06-10', '9876543212'),
  ('Sneha', 'Patel', 'sneha@gmail.com',
  'Student', '2026-06-15', '2027-06-15', '9876543213');

4 rows affected
```

INSERT INTO Loan

(book_id, member_id, loan_date, due_date,
return_date, days_overdue, status)

VALUES

(1, 1, '2026-09-01', '2026-09-15',
'2026-09-14', 0, 'Returned'),

(2, 2, '2026-09-05', '2026-09-19',
NULL, 0, 'Borrowed'),

(3, 3, '2026-09-10', '2026-09-24',
'2026-09-26', 2, 'Returned'),

(4, 4, '2026-09-12', '2026-09-26',
NULL, 0, 'Borrowed');

```
INSERT INTO Loan
  (book_id, member_id, loan_date, due_date,
  return_date, days_overdue, status)
VALUES
  (1, 1, '2026-09-01', '2026-09-15',
  '2026-09-14', 0, 'Returned'),
  (2, 2, '2026-09-05', '2026-09-19',
  NULL, 0, 'Borrowed'),
  (3, 3, '2026-09-10', '2026-09-24',
  '2026-09-26', 2, 'Returned'),
  (4, 4, '2026-09-12', '2026-09-26',
  NULL, 0, 'Borrowed');

4 rows affected
```

INSERT INTO Fine

(loan_id, member_id, fine_amount,
fine_date, payment_status)

VALUES

(3, 3, 20.00, '2026-09-26', 'Pending');

```

INSERT INTO Fine
  (loan_id, member_id, fine_amount,
   fine_date, payment_status)
VALUES
  (3, 3, 20.00, '2026-09-26', 'Pending');

```

1 rows affected

+ Code + Markdown

7.3 Constraints & Data Integrity

Constraint / Validation	Applied On	Purpose	Evidence / Screenshot Ref.
PRIMARY KEY	All entity tables	Uniquely identifies each record	Fig. 4.1
FOREIGN KEY	Loan.book_id	Maintains relationship with Book	Fig. 4.1
FOREIGN KEY	Loan.member_id	Maintains relationship with Member	Fig. 4.1
FOREIGN KEY	Fine.loan_id	Maintains relationship with Loan	Fig. 4.1
FOREIGN KEY	Fine.member_id	Maintains relationship with Member	Fig. 4.1
UNIQUE	Book.isbn	Prevents duplicate ISBN values	Fig. 4.1
UNIQUE	Member.email	Prevents duplicate member email addresses	Fig. 4.1
NOT NULL	Book.title	Ensures every book has a title	Fig. 4.1
NOT NULL	Loan.loan_date	Ensures every loan has a loan date	Fig. 4.1
CHECK	Book.available_copies	Prevents invalid copy counts	Fig. 4.1
CHECK	Fine.fine_amount	Prevents negative fine amounts	Fig. 4.1
DEFAULT	Loan.status	Automatically assigns a default loan status	Fig. 4.1

7.4 SQL Operations & Queries

DQL – Basic Retrieval Queries

Purpose of the Query	SQL Statement
Display all books available in the library	SELECT * FROM Book;
Display books belonging to the Database genre	SELECT book_id, title, genre FROM Book WHERE genre = 'Database';
Display books published after 2020	SELECT book_id, title, publication_year FROM Book WHERE publication_year > 2020;
Display all members in alphabetical order	SELECT member_id, first_name, last_name, email FROM Member ORDER BY first_name ASC;

Join Operations

Type of Join	SQL Statement
Natural Join	SELECT book_id, title, publisher_name FROM Book NATURAL JOIN Publisher;
Equi Join	SELECT b.book_id, b.title, p.publisher_name FROM Book b, Publisher p WHERE b.publisher_id = p.publisher_id;
Inner Join	SELECT l.loan_id, m.member_id, m.first_name, m.last_name, l.loan_date, l.due_date FROM Loan l INNER JOIN Member m ON l.member_id = m.member_id;
Left Outer Join	SELECT p.publisher_id, p.publisher_name, b.book_id, b.title FROM Publisher p LEFT OUTER JOIN Book b ON p.publisher_id = b.publisher_id;
Right Outer Join	SELECT b.book_id, b.title, p.publisher_id, p.publisher_name FROM Book b RIGHT OUTER JOIN Publisher p ON b.publisher_id = p.publisher_id;
Full Outer Join	SELECT b.book_id, b.title, p.publisher_id, p.publisher_name FROM Book b FULL OUTER JOIN Publisher p ON b.publisher_id = p.publisher_id;

Aggregate Functions, GROUP BY and HAVING

Function Used	SQL Statement
COUNT	SELECT COUNT(*) AS total_books FROM Book;
SUM	SELECT SUM(total_copies) AS total_copies FROM Book;
AVG	SELECT AVG(fine_amount) AS average_fine FROM Fine;
MIN and MAX	SELECT MIN(fine_amount) AS minimum_fine, MAX(fine_amount) AS maximum_fine FROM Fine;
GROUP BY + HAVING	SELECT member_id, COUNT(*) AS total_loans FROM Loan GROUP BY member_id HAVING COUNT(*) > 1;

Nested and Correlated Subqueries

Type (Nested / Correlated)	SQL Statement
Nested	SELECT book_id, title, publication_year FROM Book WHERE publication_year > (SELECT AVG(publication_year) FROM Book);
Nested	SELECT member_id, first_name, last_name FROM Member WHERE member_id IN

	(SELECT member_id FROM Loan);
Nested	SELECT book_id, title FROM Book WHERE publisher_id IN (SELECT publisher_id FROM Publisher);
Correlated	SELECT m.member_id, m.first_name, m.last_name FROM Member m WHERE EXISTS (SELECT 1 FROM Loan l WHERE l.member_id = m.member_id);
Correlated	SELECT b.book_id, b.title FROM Book b WHERE EXISTS (SELECT 1 FROM Book_Author ba WHERE ba.book_id = b.book_id);

Views and Relational Set Operations

View / Set Operation	SQL Statement
Updatable View – CREATE	CREATE VIEW Book_View AS SELECT book_id, title, isbn, genre, publication_year FROM Book;
Non-updatable View – CREATE	CREATE VIEW Loan_Details_View AS SELECT l.loan_id, b.title, m.first_name, m.last_name, l.loan_date, l.due_date, l.status FROM Loan l JOIN Book b ON l.book_id = b.book_id JOIN Member m ON l.member_id = m.member_id;
UNION	SELECT first_name, last_name FROM Member UNION SELECT first_name, last_name FROM Author;
INTERSECT	SELECT first_name, last_name FROM Member INTERSECT SELECT first_name, last_name FROM Author;
EXCEPT	SELECT first_name, last_name FROM Member EXCEPT SELECT first_name, last_name FROM Author;

DCL and TCL Operations

Command	Purpose	SQL Statement
GRANT	Give SELECT privilege on the Book table	GRANT SELECT ON Book TO library_user;
REVOKE	Remove SELECT privilege from the Book table	REVOKE SELECT ON Book FROM library_user;
COMMIT	Permanently save transaction changes	BEGIN; UPDATE Book SET available_copies = available_copies - 1 WHERE book_id = 1; COMMIT;
SAVEPOINT	Create an intermediate point in a transaction	BEGIN; SAVEPOINT book_update;
ROLLBACK TO SAVEPOINT	Undo changes made after the savepoint	ROLLBACK TO SAVEPOINT book_update; COMMIT;
ROLLBACK	Cancel all changes in the current transaction	BEGIN; UPDATE Book SET available_copies = available_copies - 1 WHERE book_id = 1; ROLLBACK;

Sample Outputs / Screenshots

Figure 1: Book records / basic SELECT output



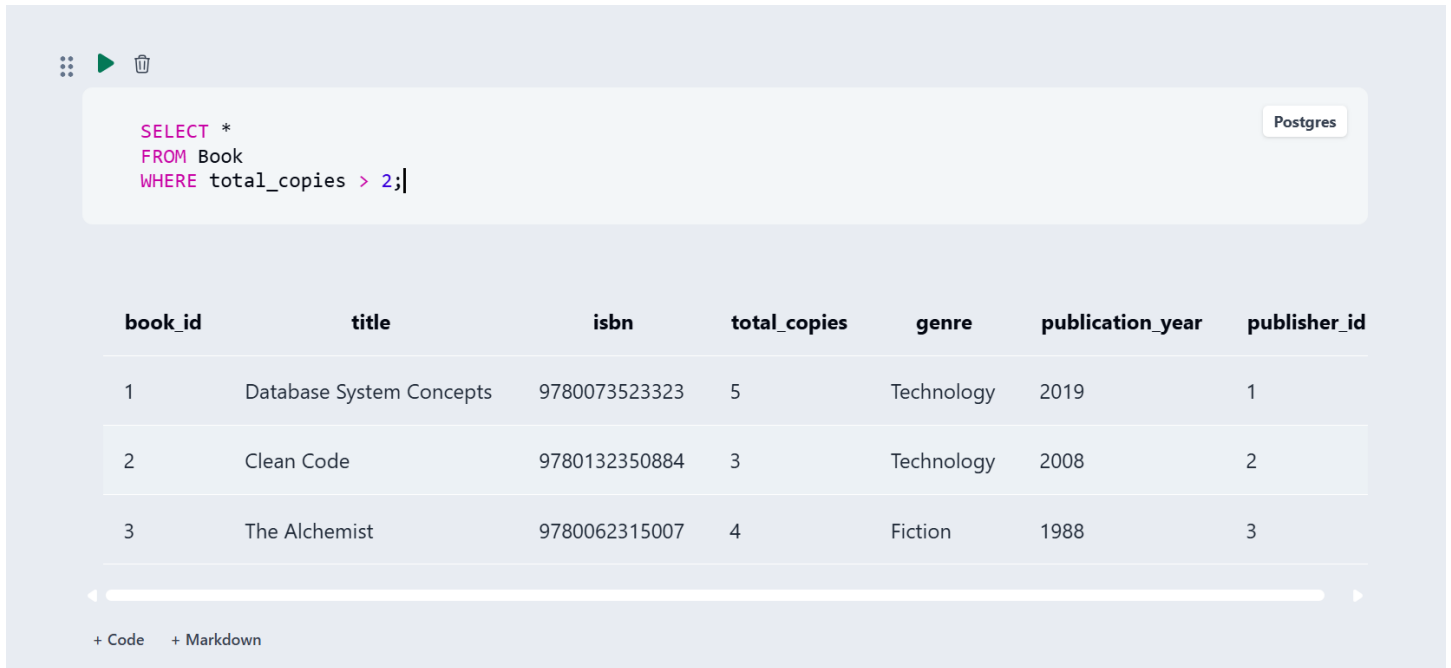
```
SELECT * FROM Book;
```

Postgres

book_id	title	isbn	total_copies	genre	publication_year	publisher_id
1	Database System Concepts	9780073523323	5	Technology	2019	1
2	Clean Code	9780132350884	3	Technology	2008	2
3	The Alchemist	9780062315007	4	Fiction	1988	3

+ Code + Markdown

Figure 2: Filtered book records / WHERE clause output



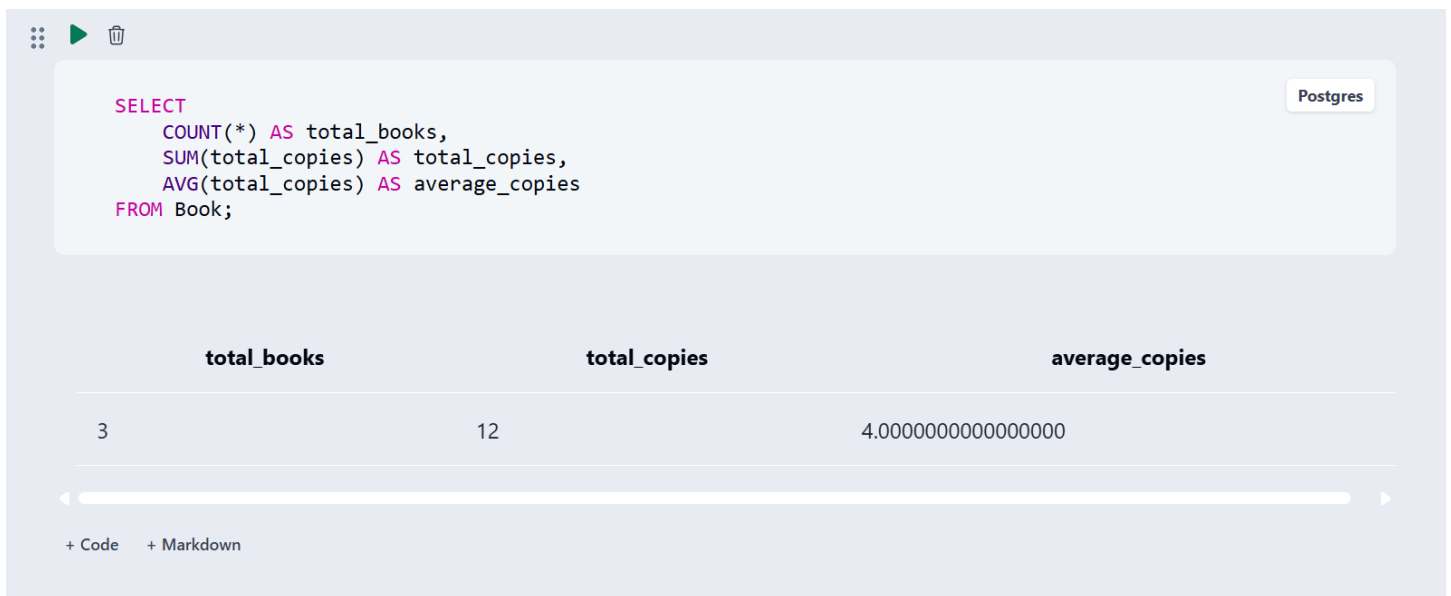
The screenshot shows a database query interface. At the top left, there are icons for a grid, a play button, and a trash can. Below these is a text area containing a SQL query: `SELECT * FROM Book WHERE total_copies > 2;`. To the right of the text area is a button labeled "Postgres". Below the text area is a table with 7 columns: **book_id**, **title**, **isbn**, **total_copies**, **genre**, **publication_year**, and **publisher_id**. The table contains 3 rows of data. Below the table is a horizontal scrollbar. At the bottom left, there are two buttons: "+ Code" and "+ Markdown".

```
SELECT *
FROM Book
WHERE total_copies > 2;
```

book_id	title	isbn	total_copies	genre	publication_year	publisher_id
1	Database System Concepts	9780073523323	5	Technology	2019	1
2	Clean Code	9780132350884	3	Technology	2008	2
3	The Alchemist	9780062315007	4	Fiction	1988	3

+ Code + Markdown

Figure 3: Aggregate functions output



The screenshot shows a database query interface. At the top left, there are icons for a grid, a play button, and a trash can. Below these is a text area containing a SQL query: `SELECT COUNT(*) AS total_books, SUM(total_copies) AS total_copies, AVG(total_copies) AS average_copies FROM Book;`. To the right of the text area is a button labeled "Postgres". Below the text area is a table with 3 columns: **total_books**, **total_copies**, and **average_copies**. The table contains 1 row of data. Below the table is a horizontal scrollbar. At the bottom left, there are two buttons: "+ Code" and "+ Markdown".

```
SELECT
COUNT(*) AS total_books,
SUM(total_copies) AS total_copies,
AVG(total_copies) AS average_copies
FROM Book;
```

total_books	total_copies	average_copies
3	12	4.0000000000000000

+ Code + Markdown

Figure 4 Member–Loan–Book JOIN output

Postgres

```

SELECT
  m.member_id,
  m.first_name,
  m.last_name,
  b.title,
  l.loan_date,
  l.due_date,
  l.return_date,
  l.status
FROM Member m
JOIN Loan l
  ON m.member_id = l.member_id
JOIN Book b
  ON l.book_id = b.book_id;

```

member_id	first_name	last_name	title	loan_date	due_date	return_date	stat
1	RISHIKA	A	Database System Concepts	"2026-07- 01T00:00:00.000Z"	"2026-07- 15T00:00:00.000Z"	"2026-07- 13T00:00:00.000Z"	Retur

```

FROM Member m
JOIN Loan l
  ON m.member_id = l.member_id
JOIN Book b
  ON l.book_id = b.book_id;

```

member_id	first_name	last_name	title	loan_date	due_date	return_date	stat
1	RISHIKA	A	Database System Concepts	"2026-07- 01T00:00:00.000Z"	"2026-07- 15T00:00:00.000Z"	"2026-07- 13T00:00:00.000Z"	Retur
2	KIRAN	R	Clean Code	"2026-07- 10T00:00:00.000Z"	"2026-07- 24T00:00:00.000Z"	null	Active
3	KRISH	A	The Alchemist	"2026-07- 20T00:00:00.000Z"	"2026-08- 03T00:00:00.000Z"	"2026-08- 10T00:00:00.000Z"	Retur

8. MODULES & FUNCTIONALITIES

S.No	Module Name	Functionality	Tables Involved
1	Book Management Module	Stores and manages book details such as title, ISBN, genre, publication year, total copies and available copies.	Book, Publisher, Book_Author
2	Author Management Module	Stores author information such as author name and nationality and maintains the relationship between authors and books.	Author, Book_Author, Book
3	Publisher Management Module	Maintains publisher details such as publisher name, country and website and connects publishers with books.	Publisher, Book
4	Member Management Module	Stores and manages registered member details including name, email, membership type, membership dates and phone number.	Member
5	Loan Management Module	Manages book borrowing and returning information, including loan date, due date, return date, overdue days and loan status.	Loan, Book, Member
6	Fine Management Module	Maintains fine information associated with loans and members, including fine amount, fine date and payment status.	Fine, Loan, Member
7	Book–Author Relationship Module	Maintains the many-to-many relationship between books and authors using a junction table.	Book_Author, Book, Author
8	Database Query and Reporting Module	Retrieves and analyzes library information using SELECT queries, joins, aggregate functions, subqueries, views and set operations.	Book, Author, Publisher, Member, Loan, Fine, Book_Author

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Repository Link:

<https://github.com/rishika1527-stack/Library-management-system-dbms..git>

Student Name	Roll Number	Role / Responsibility	Contribution Status
G.Ramababu	25B11AI347	ER Diagram and Database Schema	Completed
G.Gayathri	25B11AI348	DDL – Table Creation and ConstraintS	Completed
B.Taraka Ram	25B11AI092	DML, CRUD Operations, Joins and Aggregate Functions	Completed
Ch.Siva Rama Teja	25B11AI220	Documentation and Screenshots	Completed

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

1. Front-End Application

A web or desktop-based front-end application can be developed so that librarians and members can interact with the database through forms and user-friendly screens instead of directly executing SQL queries.

2. Stored Procedures and Triggers

Stored procedures and triggers can be introduced to automate operations such as loan processing, fine calculation, book availability updates and validation of database changes.

3. Indexing for Performance

Indexes can be created on frequently searched columns such as book title, ISBN, member ID and loan ID to improve query performance when the database grows.

4. Security and Access Control

User authentication and role-based access control can be added so that administrators, librarians and members receive different levels of database access.

5. Backup and Recovery

A systematic backup and recovery mechanism can be implemented to protect library records from accidental data loss and support restoration of the database when required.

6. Reporting and Analytics

Additional reports and dashboards can be developed to analyze popular books, borrowing trends, overdue loans, fines collected and member activity.

7. Additional Library Modules

Future versions can include modules for book reservations, notifications, inventory management, barcode or QR-code scanning and automated overdue reminders.

These enhancements would extend the current database-focused system into a more complete and automated Library Management System. The report itself identifies future improvements including additional modules, a front-end application, stored procedures and triggers, indexing, security and access control, backup and recovery, and reporting and analytics.

11. CONCLUSION

The **Library Management System** was successfully designed and implemented as a relational database for managing library information in a structured manner. The database manages information related to **books, authors, publishers, members, loans and fines**, along with the Book_Author junction table for maintaining the relationship between books and authors.

The project achieved the objective of designing a normalized relational database with appropriate **primary keys, foreign keys and integrity constraints**. The database was normalized up to **Third Normal Form (3NF)** to reduce data redundancy and maintain consistency.

SQL operations were implemented for **CREATE, INSERT, UPDATE, DELETE and SELECT**, along with joins, aggregate functions, GROUP BY, HAVING, nested and correlated subqueries, views and set operations. These operations

allow library records to be efficiently stored, modified and retrieved.

The database also establishes relationships between books, authors, publishers, members, loans and fines using relational keys. Constraints such as UNIQUE, NOT NULL, primary keys and foreign keys help maintain data integrity.

Overall, the project provides a structured database foundation for managing library records and demonstrates the practical application of DBMS concepts, relational database design, normalization and SQL operations.

12. REFERENCES

- Uploaded project DDL script: `ddl.sql`.
- Uploaded project DML/sample-data script: `dml.sql`.
- Uploaded SQL query script: `queries.sql`.
- Aditya University — DBMS Capstone Project documentation template.
- GitHub repository: **Library Management System – DBMS** (project repository).
- Uploaded ER diagram: `er-diagram.png`.
- Uploaded SQL query output screenshots: `query1_output.png` and `query2_output.png`.

1. • Uploaded project documentation: `Project_Documentation.docx`.

GitHub: <https://github.com/gollapalligayathri07-gif/LibraryManagement-System.git>

APPENDIX

A. Complete SQL Script

The final SQL implementation is supplied through the uploaded project files:

- ddl.sql — database table definitions, primary keys, foreign keys, and constraints.
- dml.sql — sample data and data manipulation operations for the library database.
- queries.sql — representative SQL operations, joins, aggregate functions, filtering, sorting, and view-related queries.
- library_management.sql — complete SQL script containing the Library Management System database implementation.

B. Additional Screenshots / Outputs

Additional evidence is provided through SQL execution screenshots showing:

- Book records / basic SELECT output.
- Filtered book records using the WHERE clause.
- Aggregate functions such as COUNT(), SUM(), and AVG().
- Member–Loan–Book JOIN output.
- Active loan records using filtering and JOIN operations.

The screenshots are included in the project repository as:

- query1_output.png
- query2_output.png

These outputs provide evidence of the SQL queries and database operations implemented in the **Library Management System**.